



Disciplina: **FGA0314 – Testes de Software**

Critérios de Testes: Caixa Branca

Elaine Venson

elainevenson@unb.br

Agenda

- Teste caixa branca -> teste estrutural
 - Grafos de Fluxo de Controle (GFC)
 - Critérios de fluxo de controle
 - Critérios baseados em complexidade
- Teste caixa cinza



Testes Caixa Branca

- Também denominado **Teste Estrutural**
- Teste baseado na **informação obtida a partir do código fonte** do produto ou componente de software
- Relacionado ao grau de **cobertura** do código fonte
- Teoricamente, **cada caminho possível** ao longo do código deveria ser testado, mas isso só pode ser feito em unidades muito simples
 - Teste exaustivo



Testes Caixa Branca

- Grafos de Fluxo de Controle (GFC) ou Grafos de Programa são utilizados pela maioria dos critérios de teste caixa branca
 - Cada vértice representa um bloco indivisível de comandos
 - Cada aresta representa um possível desvio de um bloco para outro

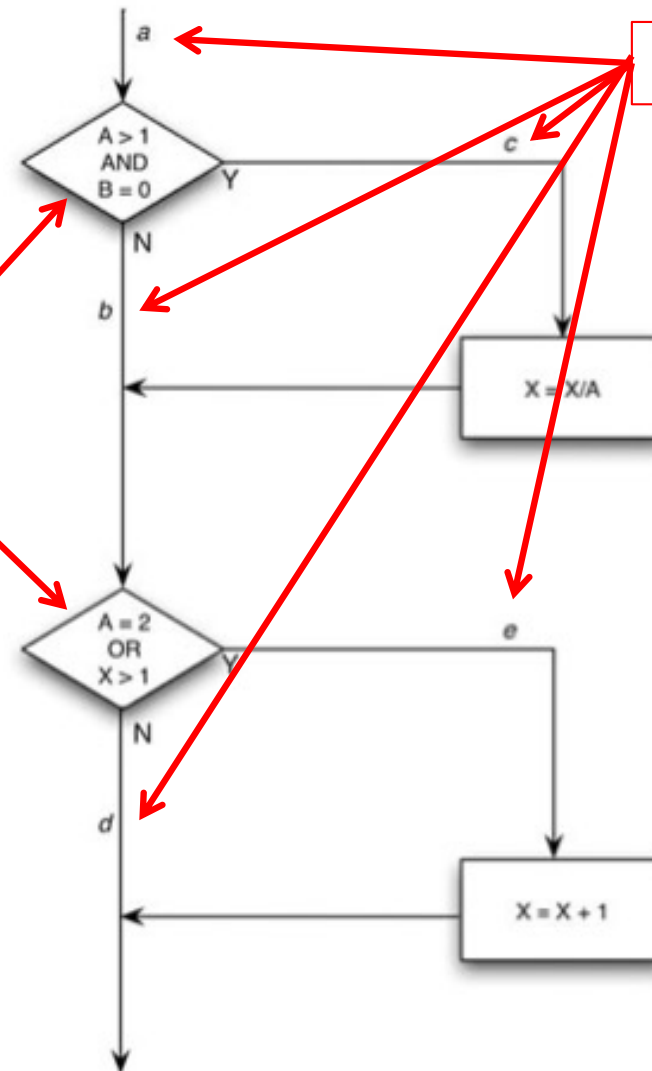


Grafo de Fluxo de Controle

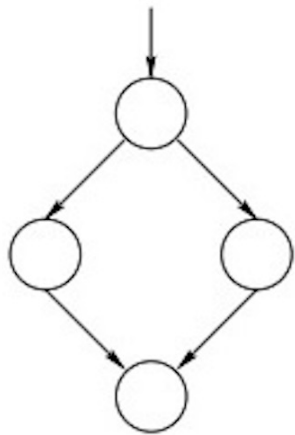
```
public void foo(int A, int B, int X) {  
    if(A>1 && B==0) {  
        X=X/A;  
    }  
    if(A==2 || X>1) {  
        X=X+1;  
    }  
}
```

Vértices
ou Nós

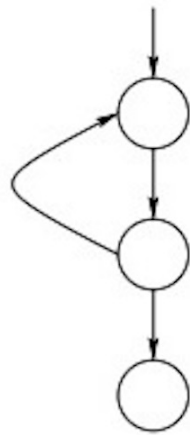
Arestas



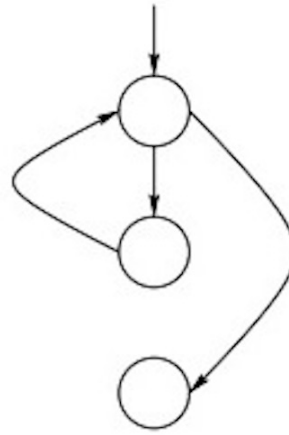
GFC - Notação



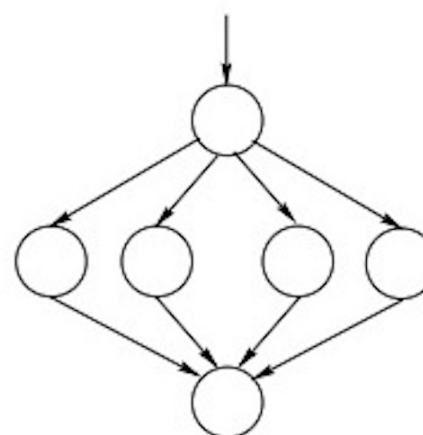
if-then-else



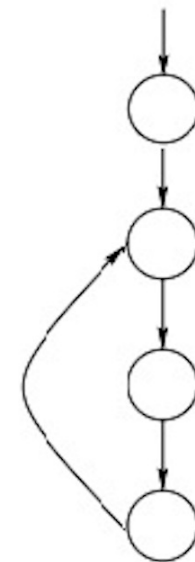
do until



while



case



for

Caixa Branca

- Para identificar todos os defeitos de um programa a partir desta abordagem seria necessário um **teste exaustivo de caminhos** -> não é possível na prática
 - Testar todos os possíveis caminhos do controle de fluxo do programa
- Limitações:
 - Não garante que o software esteja de acordo com as especificações
 - Caminhos (fluxos) não implementados não são identificados
 - Não cobre erros relacionados a dados



Caixa Branca

- Por que usar?
 - Cobrir classes de defeitos distintas às demais técnicas
 - Garantir que todos os caminhos independentes do código foram testados ao menos uma vez
 - Exercitar as decisões lógicas com valores verdadeiros e falsos
 - Executar todos os loops
 - Exercitar a estrutura de dados para garantir sua validade
- Tipicamente realizado no nível de teste unitário



Caixa Branca - Critérios

- Critérios baseados no fluxo de controle
 - Cobertura de instruções
 - Cobertura de decisões
 - Cobertura de condições
 - Cobertura de decisões/condições
 - Cobertura de condições/decisões modificada
 - Cobertura de múltiplas condições
- Critérios baseados em complexidade
 - Critério de McCabe
 - Caminhos Básicos



Cobertura de Instruções

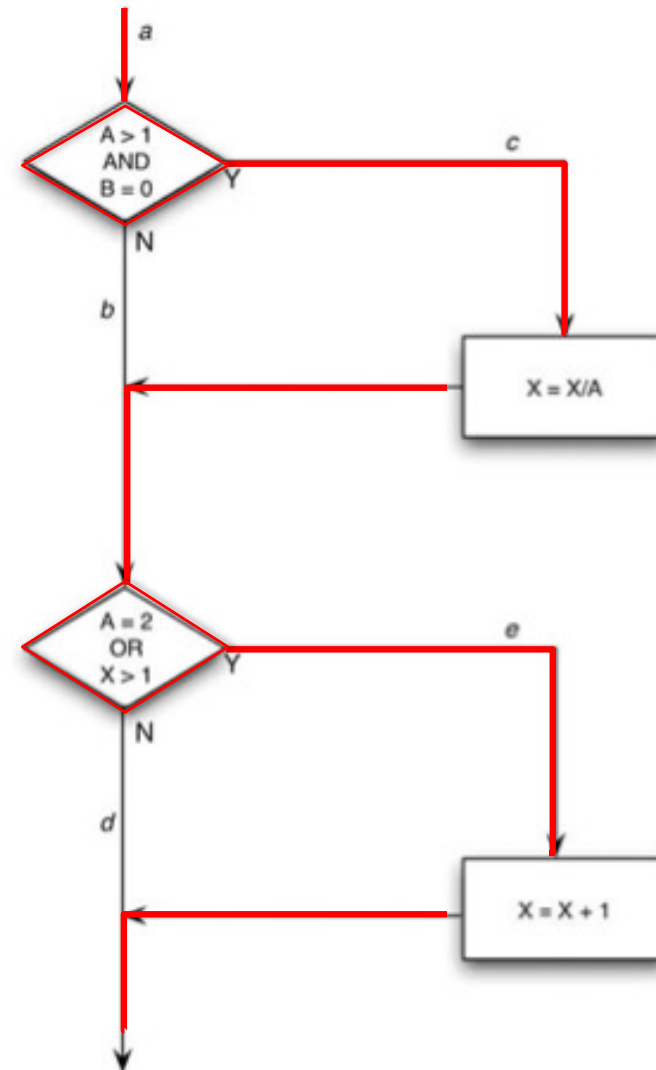
- Ou Todos-Nós ou *Statement Coverage* ou “Cobertura de Linha”
- Visa executar ao menos uma vez todas as instruções do programa



Cobertura de Instruções

```
public void foo(int A, int B, int X) {  
    if(A > 1 && B == 0) {  
        X = X / A;  
    }  
    if(A == 2 || X > 1) {  
        X = X + 1;  
    }  
}
```

Cobertura Todos-Nós



Cobertura de Instruções

- Considerado um método fraco, pois o simples fato de conseguir executar todas as instruções não aponta uma série de possíveis defeitos
- Exemplos:
 - Se uma condição com AND deveria ser OR
 - Se uma condição tipo $X > 1$ deveria ser $X \geq 1$
 - Se há um caminho onde nenhuma variável é alterada

Cobertura de Decisões

- Ou *Decision Coverage, Branch Coverage*
- Significa elaborar casos de teste para que cada decisão resulte em **verdadeiro** e em **falso** pelo menos uma vez
- Exemplos de estruturas de decisão:
 - If-else
 - Do-while
 - Switch-case



Cobertura de Decisões

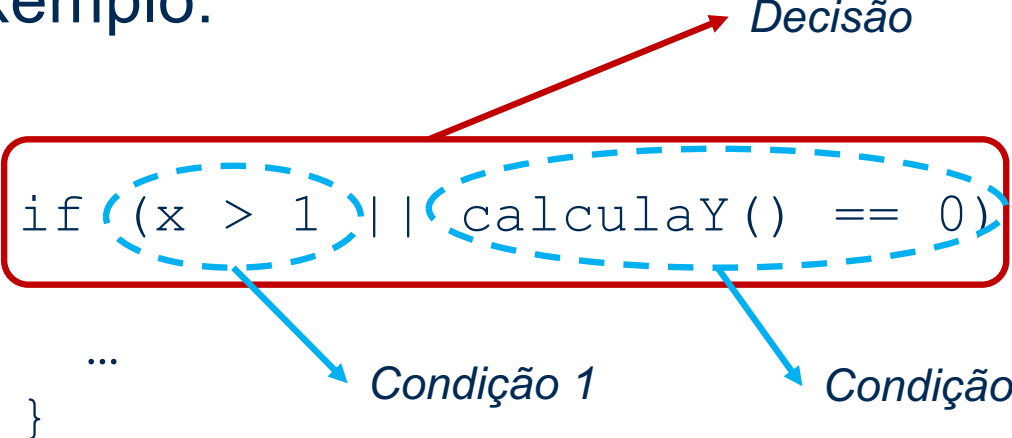
- Exemplo:

```
1  if (x > 1 || calculaY() == 0) {  
2  
3      ...  
4  }
```

Decisão

Condição 1

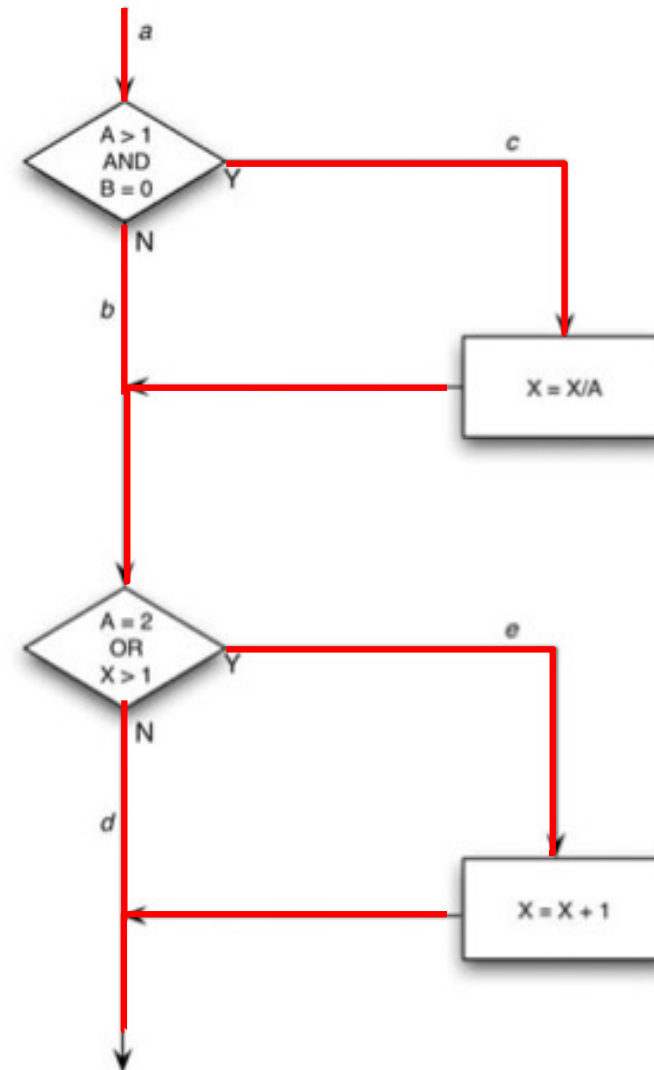
Condição 2



- Mesmo que se teste para a instrução de decisão resultar em verdadeiro e falso, a segunda condição nunca será testada quando $x > 1$

Cobertura de Decisões

```
public void foo(int A, int B, int X) {  
    if(A > 1 && B == 0) {  
        X = X/A;  
    }  
    if(A == 2 || X > 1) {  
        X = X + 1;  
    }  
}
```



Cobertura de Decisões

- É um critério mais forte que a cobertura de instruções, mas:
 - Pode não detectar defeitos como uma *condição* implementada de forma errada
 - *Condições* dentro de uma decisão podem não ser testadas

Cobertura de Condições

- Ou *Condition Coverage*
- Significa escrever casos de teste para assegurar que cada condição assuma todos os valores de saída possíveis pelo menos uma vez
- Exemplo:
 - `While ( x <= 0 && (a+b) < y) { ... }`
 - Para esta decisão, seriam elaborados casos de teste que contemplem as situações:
 - $X \leq 0$
 - $X > 0$
 - $a + b < y$
 - $a + b \geq y$



Cobertura de Condições

- Exemplo:

- `while ( x <= 0 && (a+b) < y ) { ... }`

- Para esta decisão seriam elaborados casos de teste para as situações:

- `x <= 0`

- `x > 0`

- `(a+b) < y`

- `(a+b) >= y`

- Combinações:

- `x <= 0 && (a+b) >= y (FALSO)`

- `x > 0 && (a+b) < y (FALSO)`



Cobertura de Condições

- A cobertura de condições nem sempre cobre todas as decisões
 - Dependendo da combinação utilizada, podem existir casos de teste que cubram todas as condições, mas não todas as possibilidades da decisão

Cobertura de Decisões/Condições

- Ou *decision/condition coverage*
- Significa elaborar casos de teste para que:
 - Cada condição em uma decisão assuma todos possíveis resultados pelo menos uma vez
 - Cada decisão assuma todos os possíveis resultados pelo menos uma vez
 - Cada ponto de entrada seja invocado pelo menos uma vez
- Uma fraqueza da técnica cobertura decisões/condições é que embora pareça exercitar todos os resultados de todas as condições, **há situações em que isso não ocorre porque certas condições mascaram ou bloqueiam outras condições** (em determinadas combinações de condições)
- Por exemplo, se a primeira condição em uma expressão *AND* for falsa, nenhuma das condições subsequentes na expressão precisa ser avaliada
- **Erros em expressões lógicas não são necessariamente revelados**

Cobertura de Condições/Decisões Modificada (MC/DC)

- Ou *Modified Condition / Decision Coverage (MC/DC)*
- Significa escrever casos de teste para assegurar que cada condição que afeta **independentemente** um resultado de decisão assuma todos os valores de saída possíveis pelo menos uma vez
- A independência de uma condição é mostrada provando que **apenas uma condição muda por vez**
- 100% de cobertura MC/DC implica 100% de cobertura de decisões/condições

Cobertura de Condições/Decisões Modificada (MC/DC)

- Exemplo:

- `while (  $x \leq 0$  &&  $(a+b) < y$  ) { ... }`

- Tabela Verdade

Teste	$x \leq 0$	$(a+b) < y$	Resultado
1	V	V	V
2	V	F	F
3	F	V	F
4	F	F	F

Par de independência para $x \leq 0$

- Pares selecionados:

- Para $x \leq 0 \rightarrow 1, 3$

- Para $(a+b) < y \rightarrow 1, 2$

3 combinações de condições a serem testadas (linhas 1, 2 e 3 da tabela verdade)



Cobertura de Condições/Decisões Modificada (MC/DC)

Exemplo:

- `if( A && B && C ) { ... }`
- Tabela Verdade

Teste	A	B	C	Resultado
1	V	V	V	V
2	V	V	F	F
3	V	F	V	F
4	V	F	F	F
5	F	V	V	F
6	F	V	F	F
7	F	F	V	F
8	F	F	F	F

Par de independência para A

- Pares selecionados:
 - Para A -> 1, 5

Cobertura de Condições/Decisões Modificada (MC/DC)

Exemplo:

- `if ( A && B && C ) { ... }`
- Tabela Verdade

Teste	A	B	C	Resultado
1	V	V	V	V
2	V	V	F	F
3	V	F	V	F
4	V	F	F	F
5	F	V	V	F
6	F	V	F	F
7	F	F	V	F
8	F	F	F	F

← Par de independência para B

- Pares selecionados:
 - Para A -> 1, 5
 - Para B -> 1, 3

Cobertura de Condições/Decisões Modificada (MC/DC)

Exemplo:

- `if( A && B && C ) { ... }`
- Tabela Verdade

Teste	A	B	C	Resultado
1	V	V	V	V
2	V	V	F	F
3	V	F	V	F
4	V	F	F	F
5	F	V	V	F
6	F	V	F	F
7	F	F	V	F
8	F	F	F	F

← ← Par de independência para C

- Pares selecionados:
 - Para A -> 1, 5
 - Para B -> 1, 3
 - Para C -> 1, 2
- } 4 combinações de condições (linhas 1, 2, 3 e 5)

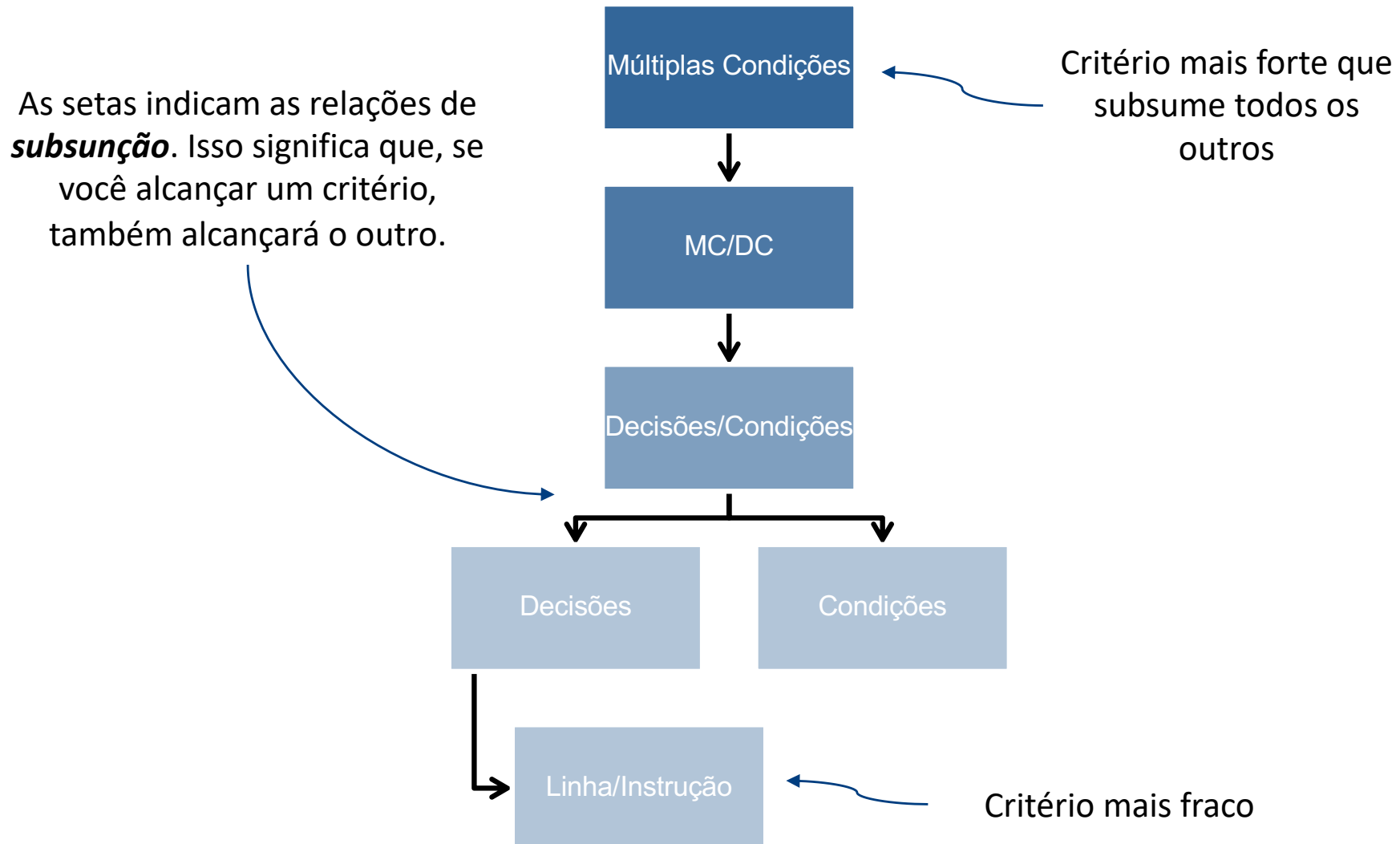
Cobertura de Condições/Decisões Modificada (MC/DC)

- O número de casos de teste no MC/DC é:
 - $N^{\circ} \text{ de casos de teste} = N^{\circ} \text{ de condições} + 1$

Cobertura de Múltiplas Condições

- Ou *multiple-condition coverage*
- Esta técnica requer que todas as possíveis **combinações** de saída de uma condição em cada decisão, bem como todos os pontos de entrada sejam exercitados ao menos uma vez
- O número de casos de teste na cobertura de múltiplas condições é:
 - $N^{\circ} \text{ de casos de teste} = 2^{\wedge} N^{\circ} \text{ de condições}$
 - Para 3 condições $\rightarrow 2^{\wedge} 3 = 8$
 - (todas as possibilidades da tabela verdade)

Critérios de cobertura e suas relações de subsunção



Exemplo: função BONUS

- Especificação da função BONUS:
 - A função BONUS adiciona \$200 ao salário de todos os funcionários no departamento ou departamentos com o maior resultado de vendas.
 - Entretanto, se o salário de um funcionário elegível for \$15.000 ou mais, ou se o funcionário for um gerente, o salário é adicionado em apenas \$100.
 - Em caso de sucesso, a função retorna 0.
 - Se ao menos uma das tabelas estiver vazia, retorna código de erro 1.
 - Se a função não encontrar funcionários em um departamento elegível, retorna código de erro 2.



Exemplo: função BONUS

```

BONUS : PROCEDURE (EMPTAB,DEPTTAB,ESIZE,DSIZE,ERRCODE);
DECLARE 1  EMPTAB (*),
        2  NAME CHAR(6),
        2  CODE CHAR(1),
        2  DEPT CHAR(3),
        2  SALARY FIXED DECIMAL(7,2);
DECLARE 1  DEPTTAB (*),
        2  DEPT CHAR(3),
        2  SALES FIXED DECIMAL(8,2);
DECLARE (ESIZE,DSIZE) FIXED BINARY;
DECLARE ERRCODE FIXED DECIMAL(1);
DECLARE MAXSALES FIXED DECIMAL(8,2) INIT(0); /*MAX. SALES IN DEPTTAB*/
DECLARE (I,J,K) FIXED BINARY;               /*COUNTERS*/
DECLARE FOUND BIT(1); /*TRUE IF ELIGIBLE DEPT. HAS EMPLOYEES*/
DECLARE SINC FIXED DECIMAL(7,2) INIT(200.00); /*STANDARD INCREMENT*/
DECLARE LINC FIXED DECIMAL(7,2) INIT(100.00); /*LOWER INCREMENT*/
DECLARE LSALARY FIXED DECIMAL(7,2) INIT(15000.00); /*SALARY BOUNDARY*/
DECLARE MGR CHAR(1) INIT('M');
    
```

Name	Job code	Dept.	Salary

Employee table

Dept.	Sales

Department table

(continued)



Exemplo: função BONUS

Especificação da função BONUS:

- Adiciona \$2.000 ao salário de todos os funcionários no departamento ou departamentos com o maior resultado de vendas.
- Se o salário de um funcionário elegível for \$15.000 ou mais, ou se o funcionário for um gerente, o salário é adicionado em apenas \$1.000.
- Retorno:
 - 0: sucesso.
 - 1: se ao menos uma das tabelas estiver vazia.
 - 2: se a função não encontrar funcionários em um departamento elegível.

```
1  ERRCODE=0;
2  IF(ESIZE<=0) | (DSIZE<=0)
3      THEN ERRCODE=1;                /*EMPTAB OR DEPTTAB ARE EMPTY*/
4  ELSE DO;
5      DO I = 1 TO DSIZE;              /*FIND MAXSALES AND MAXDEPTS*/
6          IF(SALES(I)>=MAXSALES) THEN MAXSALES=SALES(I);
7      END;
8      DO J = 1 TO DSIZE;
9          IF(SALES(J)=MAXSALES)        /*ELIGIBLE DEPARTMENT*/
10             THEN DO;
11                 FOUND='0'B;
12                 DO K = 1 TO ESIZE;
13                     IF(EMPTAB.DEPT(K)=DEPTTAB.DEPT(J))
14                         THEN DO;
15                             FOUND='1'B;
16                             IF(SALARY(K)>=LSALARY) | CODE(K)=MGR)
17                                 THEN SALARY(K)=SALARY(K)+LINC;
18                                 ELSE SALARY(K)=SALARY(K)+SINC;
19                             END;
20                         END;
21                     IF(-FOUND) THEN ERRCODE=2;
22                 END;
23             END;
24  END;
25 END;
```



Exemplo: função BONUS

- Primeiro passo: listar as decisões do programa

```
1  ERRCODE=0;
2  IF(ESIZE<=0) | (DSIZE<=0)
3      THEN ERRCODE=1;          /*EMPTAB OR DEPTTAB ARE EMPTY*/
4      ELSE DO;
5          DO I = 1 TO DSIZE;    /*FIND MAXSALES AND MAXDEPTS*/
6              IF(SALES(I)>=MAXSALES) THEN MAXSALES=SALES(I);
7          END;
8          DO J = 1 TO DSIZE;
9              IF(SALES(J)=MAXSALES)          /*ELIGIBLE DEPARTMENT*/
10                 THEN DO;
11                     FOUND='0'B;
12                     DO K = 1 TO ESIZE;
13                         IF(EMPTAB.DEPT(K)=DEPTTAB.DEPT(J))
14                             THEN DO;
15                                 FOUND='1'B;
16                                 IF(SALARY(K)>=LSALARY) | CODE(K)=MGR)
17                                     THEN SALARY(K)=SALARY(K)+LINC;
18                                     ELSE SALARY(K)=SALARY(K)+SINC;
19                                 END;
20                             END;
21                         IF(-FOUND) THEN ERRCODE=2;
22                     END;
23                 END;
24             END;
25 END;
```

Obs.: as instruções DO não precisam de atenção especial, uma vez que qualquer caso de teste que faça com que uma instrução DO seja executada fará com que ela se ramifique em ambas as direções (ou seja, entre no corpo do loop e pule o corpo do loop).

Relembrando: critérios caixa branca

- Critérios baseados no fluxo de controle
 - Cobertura de instruções
 - Cobertura de decisões
 - Cobertura de condições
 - Cobertura de decisões-condições
 - Cobertura de múltiplas condições



Exemplo: função BONUS

Cobertura de decisões

```

1  ERRCODE=0;
2  IF(ESIZE<=0) | (DSIZE<=0)
3      THEN ERRCODE=1;          /*EMPTAB OR DEPTTAB ARE EMPTY*/
4      ELSE DO;
5          DO I = 1 TO DSIZE;    /*FIND MAXSALES AND MAXDEPTS*/
6              IF(SALES(I)>=MAXSALES) THEN MAXSALES=SALES(I);
7          END;
8          DO J = 1 TO DSIZE;
9              IF(SALES(J)=MAXSALES)      /*ELIGIBLE DEPARTMENT*/
10                 THEN DO;
11                     FOUND='0'B;
12                     DO K = 1 TO ESIZE;
13                         IF(EMPTAB.DEPT(K)=DEPTTAB.DEPT(J))
14                             THEN DO;
15                                 FOUND='1'B;
16                                 IF (SALARY(K)>=LSALARY) | CODE(K)=MGR)
17                                     THEN SALARY(K)=SALARY(K)+LINC;
18                                     ELSE SALARY(K)=SALARY(K)+SINC;
19                                 END;
20                             END;
21                                 IF(-FOUND) THEN ERRCODE=2;
22                             END;
23                         END;
24                     END;
25 END;

```

Decisão	Situação para True	Situação para False
2	ESIZE ou DSIZE <= 0	ESIZE e DSIZE > 0
6	Vai sempre ocorrer pelo menos uma vez	Ordenar a tabela DEPTTAB para que um departamento com menor resultado de vendas venha após um com maior resultado
9	Vai sempre ocorrer pelo menos uma vez	Os departamentos não tem todos o mesmo valor de vendas
13	Há um empregado em um departamento elegível	Há um empregado em um departamento não elegível
16	Um empregado de dept. elegível é um gerente ou recebe LSALARY ou mais	Um empregado de dept. elegível não é um gerente e recebe menos que LSALARY
21	Um departamento elegível não contém empregados	Um departamento elegível contém ao menos um empregado

Exemplo: função BONUS

Cobertura de decisões

- Elaborar casos de teste suficientes para invocar os dois resultados de cada uma das seis decisões.
- Como duas situações vão sempre ocorrer, são 10 situações que precisam ser consideradas.

Decisão	Situação para True	Situação para False
2	ESIZE ou DSIZE $\leq$ 0	ESIZE e DSIZE $>$ 0
6	Vai sempre ocorrer pelo menos uma vez	Ordenar a tabela DEPTTAB para que um departamento com menor resultado de vendas venha após um com maior resultado
9	Vai sempre ocorrer pelo menos uma vez	Os departamentos não tem todos o mesmo valor de vendas
13	Há um empregado em um departamento elegível	Há um empregado de um departamento não elegível
16	Um empregado de dept. elegível é um gerente ou recebe LSALARY ou mais	Um empregado de dept. elegível não é um gerente e recebe menos que LSALARY
21	Um departamento elegível não contém empregados	Um departamento elegível contém ao menos um empregado



Exemplo: função BONUS

Cobertura de decisões

D	Situação para True	Situação para False
2	ESIZE ou DSIZE ≤ 0	ESIZE e DSIZE > 0
6	Vai sempre ocorrer pelo menos uma vez	Ordenar a tabela DEPTTAB para que um departamento com menor resultado de vendas venha após um com maior resultado
9	Vai sempre ocorrer pelo menos uma vez	Os departamentos não tem todos o mesmo valor de vendas
13	Há um empregado em um departamento elegível	Há um empregado de um departamento não elegível
16	Um empregado de dept. elegível é um gerente ou recebe LSALARY ou mais	Um empregado de dept. elegível não é um gerente e recebe menos que LSALARY
21	Um departamento elegível não contém empregados	Um departamento elegível contém ao menos um empregado

Test case	Input	Expected output																														
1	ESIZE = 0 All other inputs are irrelevant	ERRCODE = 1 ESIZE, DSIZE, EMPTAB, and DEPTTAB are unchanged																														
2	ESIZE = DSIZE = 3 EMPTAB <table><tr><td>JONES</td><td>E</td><td>D42</td><td>21,000.00</td></tr><tr><td>SMITH</td><td>E</td><td>D32</td><td>14,000.00</td></tr><tr><td>LORIN</td><td>E</td><td>D42</td><td>10,000.00</td></tr></table> DEPTTAB <table><tr><td>D42</td><td>10,000.00</td></tr><tr><td>D32</td><td>8,000.00</td></tr><tr><td>D95</td><td>10,000.00</td></tr></table>	JONES	E	D42	21,000.00	SMITH	E	D32	14,000.00	LORIN	E	D42	10,000.00	D42	10,000.00	D32	8,000.00	D95	10,000.00	ERRCODE = 2 ESIZE, DSIZE, and DEPTTAB are unchanged EMPTAB <table><tr><td>JONES</td><td>E</td><td>D42</td><td>21,100.00</td></tr><tr><td>SMITH</td><td>E</td><td>D32</td><td>14,000.00</td></tr><tr><td>LORIN</td><td>E</td><td>D42</td><td>10,200.00</td></tr></table>	JONES	E	D42	21,100.00	SMITH	E	D32	14,000.00	LORIN	E	D42	10,200.00
JONES	E	D42	21,000.00																													
SMITH	E	D32	14,000.00																													
LORIN	E	D42	10,000.00																													
D42	10,000.00																															
D32	8,000.00																															
D95	10,000.00																															
JONES	E	D42	21,100.00																													
SMITH	E	D32	14,000.00																													
LORIN	E	D42	10,200.00																													

Situações não exploradas por estes casos de teste:

- Retorno = 0
- Empregado é gerente
- Tabela de departamentos vazia



Exemplo: função BONUS

Cobertura de condições

```

1  ERRCODE=0;
2  IF(ESIZE<=0)|(DSIZE<=0)
3      THEN ERRCODE=1;          /*EMPTAB OR DEPTTAB ARE EMPTY*/
4      ELSE DO;
5          DO I = 1 TO DSIZE;      /*FIND MAXSALES AND MAXDEPTS*/
6              IF(SALES(I)>=MAXSALES) THEN MAXSALES=SALES(I);
7          END;
8          DO J = 1 TO DSIZE;
9              IF(SALES(J)=MAXSALES)      /*ELIGIBLE DEPARTMENT*/
10                 THEN DO;
11                     FOUND='0'B;
12                     DO K = 1 TO ESIZE;
13                         IF(EMPTAB.DEPT(K)=DEPTTAB.DEPT(J))
14                             THEN DO;
15                                 FOUND='1'B;
16                                 IF (SALARY(K)>=LSALARY) | CODE(K)=MGR)
17                                     THEN SALARY(K)=SALARY(K)+LINC;
18                                     ELSE SALARY(K)=SALARY(K)+SINC;
19                                 END;
20                             END;
21                                 IF(-FOUND) THEN ERRCODE=2;
22                             END;
23                         END;
24                     END;
25 END;

```

D	Condição	Situação para True	Situação para False
2	ESIZE <= 0	ESIZE <= 0	ESIZE > 0
2	DSIZE <= 0	DSIZE <= 0	DSIZE > 0
6	SALES(I) >= MAXSALES	Vai sempre ocorrer pelo menos uma vez	Ordenar a tabela DEPTTAB para que um departamento com menor resultado de vendas venha após um com maior resultado
9	SALES(J)=MAXSALES	Vai sempre ocorrer pelo menos uma vez	Os departamentos não tem todos o mesmo valor de vendas
13	EMPTAB.DEPT(K)=DEPTTAB.DEPT(J)	Há um empregado em um departamento elegível	Há um empregado de um departamento não elegível
16	SALARY(K) >= LSALARY	Um empregado de dept. elegível recebe LSALARY ou mais	Um empregado de dept. elegível recebe menos que LSALARY
16	CODE(K)=MGR	Um empregado de dept. elegível é um gerente	Um empregado de dept. elegível não é um gerente
21	-FOUND	Um departamento elegível não contém empregados	Um departamento elegível contém ao menos um empregado

Exemplo: função BONUS

Cobertura de condições

- Elaborar casos de teste o suficiente para invocar os dois resultados de cada uma das condições em cada uma das decisões.
- Apenas decisões 2 e 16 possuem mais de uma condição.
- Como duas situações vão sempre ocorrer, são 14 situação que precisam ser consideradas.

D	Condição	Situação para True	Situação para False
2	ESIZE <= 0	ESIZE <= 0	ESIZE > 0
2	DSIZE <= 0	DSIZE <= 0	DSIZE > 0
6	SALES(I) >= MAXSALES	Vai sempre ocorrer pelo menos uma vez	Ordenar a tabela DEPTTAB para que um departamento com menor resultado de vendas venha após um com maior resultado
9	SALES(J)=MAXSALES	Vai sempre ocorrer pelo menos uma vez	Os departamentos não tem todos o mesmo valor de vendas
13	EMPTAB.DEPT(K)=DEPTTAB.DEPT(J)	Há um empregado em um departamento elegível	Há um emprego de um departamento não elegível
16	SALARY(K) >= LSALARY	Um empregado de dept. elegível recebe LSALARY ou mais	Um empregado de dept. elegível recebe menos que LSALARY
16	CODE(K)=MGR	Um empregado de dept. elegível é um gerente	Um empregado de dept. elegível não é um gerente
21	-FOUND	Um departamento elegível não contém empregados	Um departamento elegível contém ao menos um empregado



Exemplo: função BONUS

Cobertura de condições

D	Condição	Situação para True	Situação para False	Test case	Input	Expected output																														
2	ESIZE <= 0	ESIZE <= 0	ESIZE > 0	1	ESIZE = DSIZE = 0 All other inputs are irrelevant	ERRCODE = 1 ESIZE, DSIZE, EMPTAB, and DEPTTAB are unchanged																														
2	DSIZE <= 0	DSIZE <= 0	DSIZE > 0																																	
6	SALES(I) >= MAXSALES	Vai sempre ocorrer pelo menos uma vez	Ordenar a tabela DEPTTAB para que um departamento com menor resultado de vendas venha após um com maior resultado	2	ESIZE = DSIZE = 3 EMPTAB<table><tr><td>JONES</td><td>E</td><td>D42</td><td>21,000.00</td></tr><tr><td>SMITH</td><td>E</td><td>D32</td><td>14,000.00</td></tr><tr><td>LORIN</td><td>M</td><td>D42</td><td>10,000.00</td></tr></table> DEPTTAB<table><tr><td>D42</td><td>10,000.00</td></tr><tr><td>D32</td><td>8,000.00</td></tr><tr><td>D95</td><td>10,000.00</td></tr></table>	JONES	E	D42	21,000.00	SMITH	E	D32	14,000.00	LORIN	M	D42	10,000.00	D42	10,000.00	D32	8,000.00	D95	10,000.00	ERRCODE = 2 ESIZE, DSIZE, and DEPTTAB are unchanged EMPTAB<table><tr><td>JONES</td><td>E</td><td>D42</td><td>21,000.00</td></tr><tr><td>SMITH</td><td>E</td><td>D32</td><td>14,000.00</td></tr><tr><td>LORIN</td><td>M</td><td>D42</td><td>10,100.00</td></tr></table>	JONES	E	D42	21,000.00	SMITH	E	D32	14,000.00	LORIN	M	D42	10,100.00
JONES	E	D42	21,000.00																																	
SMITH	E	D32	14,000.00																																	
LORIN	M	D42	10,000.00																																	
D42	10,000.00																																			
D32	8,000.00																																			
D95	10,000.00																																			
JONES	E	D42	21,000.00																																	
SMITH	E	D32	14,000.00																																	
LORIN	M	D42	10,100.00																																	
9	SALES(J)=MAXSALES	Vai sempre ocorrer pelo menos uma vez	Os departamentos não tem todos o mesmo valor de vendas																																	
13	EMPTAB.DEPT(K)=DEPTTAB.DEPT(J)	Há um empregado em um departamento elegível	Há um empregado de um departamento não elegível																																	
16	SALARY(K) >= LSALARY	Um empregado de dept. elegível recebe LSALARY ou mais	Um empregado de dept. elegível recebe menos que LSALARY																																	
16	CODE(K)=MGR	Um empregado de dept. elegível é um gerente	Um empregado de dept. elegível não é um gerente																																	
21	-FOUND	Um departamento elegível não contém empregados	Um departamento elegível contém ao menos um empregado																																	

Casos de teste piores que os da cobertura de decisões:

- Não executam todas as comandos (linha 18)
- Também não testa retorno = 0
- Se a decisão 2 tivesse sido erroneamente implementada com AND em vez de OR, o defeito não teria sido detectado.



Casos de teste piores que os da cobertura de decisões:

- Não executam todas os comandos (linha 18)
- Também não testa retorno = 0
- Se a decisão 2 tivesse sido erroneamente implementada com AND em vez de OR, o defeito não teria sido detectado.



Exemplo: função BONUS

Cobertura de decisões/condições

- Eliminar a maioria das fraquezas dos casos de testes baseados apenas em condições
- Todos os resultados de todas as condições E de todas as decisões seriam considerados
- Para fazer a decisão 16 gerar os dois resultados, bastaria fazer Jones um gerente e Lorin não gerente.
- Porém, se o compilador para a avaliação de uma expressão OR quando o primeiro termo for TRUE a condição CODE(K)=MGR (linha 16) nunca teria o resultado TRUE

```
1  ERRCODE=0;
2  IF(ESIZE<=0) | (DSIZE<=0)
3      THEN ERRCODE=1;          /*EMPTAB OR DEPTTAB ARE EMPTY*/
4      ELSE DO;
5          DO I = 1 TO DSIZE;    /*FIND MAXSALES AND MAXDEPTS*/
6              IF(SALES(I)>=MAXSALES) THEN MAXSALES=SALES(I);
7          END;
8          DO J = 1 TO DSIZE;
9              IF(SALES(J)=MAXSALES)          /*ELIGIBLE DEPARTMENT*/
10                 THEN DO;
11                     FOUND='0'B;
12                     DO K = 1 TO ESIZE;
13                         IF(EMPTAB.DEPT(K)=DEPTTAB.DEPT(J))
14                             THEN DO;
15                                 FOUND='1'B;
16                                 IF(SALARY(K)>=LSALARY) | CODE(K)=MGR
17                                     THEN SALARY(K)=SALARY(K)+LINC;
18                                     ELSE SALARY(K)=SALARY(K)+SINC;
19                                 END;
20                             END;
21                         IF(-FOUND) THEN ERRCODE=2;
22                     END;
23                 END;
24             END;
25 END;
```



Exemplo: função BONUS

Cobertura MC/DC

- Elaborar casos de teste o suficiente tal que cada condição que afeta independentemente um resultado de decisão assuma todos os valores de saída possíveis pelo menos uma vez
- Partir da tabela com todas as condições:
 - Decisões 6, 9, 13 e 21 tem apenas uma condição
 - Decisões 2 e 16 tem **duas** condições -> **três** casos de teste são necessários (n° de condições + 1)

D	Condição	Situação para True	Situação para False
2	ESIZE <= 0	ESIZE <= 0	ESIZE > 0
2	DSIZE <= 0	DSIZE <= 0	DSIZE > 0
6	SALES(I) >= MAXSALES	Vai sempre ocorrer pelo menos uma vez	Ordenar a tabela DEPTTAB para que um departamento com menor resultado de vendas venha após um com maior resultado
9	SALES(J)=MAXSALES	Vai sempre ocorrer pelo menos uma vez	Os departamentos não tem todos o mesmo valor de vendas
13	EMPTAB.DEPT(K)=DEPTTAB.DEPT(J)	Há um empregado em um departamento elegível	Há um emprego de um departamento não elegível
16	SALARY(K) >= LSALARY	Um empregado de dept. elegível recebe LSALARY ou mais	Um empregado de dept. elegível recebe menos que LSALARY
16	CODE(K)=MGR	Um empregado de dept. elegível é um gerente	Um empregado de dept. elegível não é um gerente
21	-FOUND	Um departamento elegível não contém empregados	Um departamento elegível contém ao menos um empregado



Exemplo: função BONUS

Cobertura MC/DC

- Decisões 2 e 16 tem **duas** condições -> **três** casos de teste são necessários (nº de condições + 1)
- Decisão 2: 2 IF(ESIZE<=0) | (DSIZE<=0)
 - 1) V F -> V e F F -> F
 - 2) F V -> V e F F -> F
- Decisão 16: IF(SALARY(K)>=LSALARY) | CODE(K)=MGR)
 - 1) V F -> V e F F -> F
 - 2) F V -> V e F F -> F

D	Condição	Situação para True	Situação para False
2	ESIZE <= 0	ESIZE <= 0	ESIZE > 0
2	DSIZE <= 0	DSIZE <= 0	DSIZE > 0
6	SALES(I) >= MAXSALES	Vai sempre ocorrer pelo menos uma vez	Ordenar a tabela DEPTTAB para que um departamento com menor resultado de vendas venha após um com maior resultado
9	SALES(J)=MAXSALES	Vai sempre ocorrer pelo menos uma vez	Os departamentos não tem todos o mesmo valor de vendas
13	EMPTAB.DEPT(K)=DEPTTAB.DEPT(J)	Há um empregado em um departamento elegível	Há um empregado de um departamento não elegível
16	SALARY(K) >= LSALARY	Um empregado de dept. elegível recebe LSALARY ou mais	Um empregado de dept. elegível recebe menos que LSALARY
16	CODE(K)=MGR	Um empregado de dept. elegível é um gerente	Um empregado de dept. elegível não é um gerente
21	-FOUND	Um departamento elegível não contém empregados	Um departamento elegível contém ao menos um empregado

Exemplo: função BONUS

Cobertura MC/DC

- Decisão 16 -> 3 casos de teste
 - 1) V F -> V
 - Empregado de dept. elegível recebe LSALARY ou mais
 - Empregado não é gerente
 - (TOY)
 - 2) F F -> F
 - Empregado de dept. elegível recebe menos que LSALARY
 - Empregado não é gerente
 - (LORIN)
 - 2) F V -> V
 - Empregado de dept. elegível recebe menos que LSALARY
 - Empregado é gerente
 - (WARNS)

D	Condição	Situação para True	Situação para False
2	ESIZE <= 0	ESIZE <= 0	ESIZE > 0
2	DSIZE <= 0	DSIZE <= 0	DSIZE > 0
16	SALARY(K) >= LSALARY	Um empregado de dept. elegível recebe LSALARY ou mais	Um empregado de dept. elegível recebe menos que LSALARY
16	CODE(K)=MGR	Um empregado de dept. elegível é um gerente	Um empregado de dept. elegível não é um gerente

Test case	Input	Expected output																																								
1	ESIZE = 0 DSIZE > 0 All other inputs are irrelevant	ERRCODE = 1 ESIZE, DSIZE, EMPTAB, and DEPTTAB are unchanged																																								
2	ESIZE > 0 DSIZE = 0 All other inputs are irrelevant	Same as above																																								
3	ESIZE = 4 DSIZE = 4 EMPTAB <table><tr><td>WARNS</td><td>M</td><td>D95</td><td>12,000.00</td></tr><tr><td>LORIN</td><td>E</td><td>D42</td><td>10,000.00</td></tr><tr><td>TOY</td><td>E</td><td>D95</td><td>16,000.00</td></tr><tr><td>SMITH</td><td>E</td><td>D32</td><td>14,000.00</td></tr></table> DEPTTAB <table><tr><td>D42</td><td>10,000.00</td></tr><tr><td>D32</td><td>8,000.00</td></tr><tr><td>D95</td><td>10,000.00</td></tr><tr><td>D44</td><td>10,000.00</td></tr></table>	WARNS	M	D95	12,000.00	LORIN	E	D42	10,000.00	TOY	E	D95	16,000.00	SMITH	E	D32	14,000.00	D42	10,000.00	D32	8,000.00	D95	10,000.00	D44	10,000.00	ERRCODE = 2 ESIZE, DSIZE, and DEPTTAB are unchanged EMPTAB <table><tr><td>WARNS</td><td>M</td><td>D95</td><td>12,100.00</td></tr><tr><td>LORIN</td><td>E</td><td>D42</td><td>10,200.00</td></tr><tr><td>TOY</td><td>E</td><td>D95</td><td>16,100.00</td></tr><tr><td>SMITH</td><td>E</td><td>D32</td><td>14,000.00</td></tr></table>	WARNS	M	D95	12,100.00	LORIN	E	D42	10,200.00	TOY	E	D95	16,100.00	SMITH	E	D32	14,000.00
WARNS	M	D95	12,000.00																																							
LORIN	E	D42	10,000.00																																							
TOY	E	D95	16,000.00																																							
SMITH	E	D32	14,000.00																																							
D42	10,000.00																																									
D32	8,000.00																																									
D95	10,000.00																																									
D44	10,000.00																																									
WARNS	M	D95	12,100.00																																							
LORIN	E	D42	10,200.00																																							
TOY	E	D95	16,100.00																																							
SMITH	E	D32	14,000.00																																							

V F ->

F V ->

F F ->

F V ->

F F ->

V F ->

Exemplo: função BONUS

Cobertura de múltiplas condições

- Elaborar casos de teste o suficiente tal que todas as possíveis combinações de condições em cada decisão sejam invocadas pelo menos um vez.
- Partir da tabela com todas as condições:
 - Decisões 6, 9, 13 e 21 tem duas combinações cada
 - Decisões 2 e 16 tem quatro combinações cada

D	Condição	Situação para True	Situação para False
2	ESIZE <= 0	ESIZE <= 0	ESIZE > 0
2	DSIZE <= 0	DSIZE <= 0	DSIZE > 0
6	SALES(I) >= MAXSALES	Vai sempre ocorrer pelo menos uma vez	Ordenar a tabela DEPTTAB para que um departamento com menor resultado de vendas venha após um com maior resultado
9	SALES(J)=MAXSALES	Vai sempre ocorrer pelo menos uma vez	Os departamentos não tem todos o mesmo valor de vendas
13	EMPTAB.DEPT(K)=DEPTTAB.DEPT(J)	Há um empregado em um departamento elegível	Há um emprego de um departamento não elegível
16	SALARY(K) >= LSALARY	Um empregado de dept. elegível recebe LSALARY ou mais	Um empregado de dept. elegível recebe menos que LSALARY
16	CODE(K)=MGR	Um empregado de dept. elegível é um gerente	Um empregado de dept. elegível não é um gerente
21	-FOUND	Um departamento elegível não contém empregados	Um departamento elegível contém ao menos um empregado



Exemplo: função BONUS

Cobertura de múltiplas condições

D	Condição	Situação para True	Situação para False
2	ESIZE <= 0	ESIZE <= 0	ESIZE > 0
2	DSIZE <= 0	DSIZE <= 0	DSIZE > 0
6	SALES(I) >= MAXSALES	Vai sempre ocorrer pelo menos uma vez	Ordenar a tabela DEPTTAB para que um departamento com menor resultado de vendas venha após um com maior resultado
9	SALES(J)=MAXSALES	Vai sempre ocorrer pelo menos uma vez	Os departamentos não tem todos o mesmo valor de vendas
13	EMPTAB.DEPT(K)=DEPTTAB.DEPT(J)	Há um empregado em um departamento elegível	Há um emprego de um departamento não elegível
16	SALARY(K) >= LSALARY	Um empregado de dept. elegível recebe LSALARY ou mais	Um empregado de dept. elegível recebe menos que LSALARY
16	CODE(K)=MGR	Um empregado de dept. elegível é um gerente	Um empregado de dept. elegível não é um gerente
21	-FOUND	Um departamento elegível não contém empregados	Um departamento elegível contém ao menos um empregado

V V ->

Test case	Input	Expected output																																																
1	ESIZE = 0 DSIZE = 0 All other inputs are irrelevant	ERRCODE = 1 ESIZE, DSIZE, EMPTAB, and DEPTTAB are unchanged																																																
2	ESIZE = 0 DSIZE > 0 All other inputs are irrelevant	Same as above																																																
3	ESIZE > 0 DSIZE = 0 All other inputs are irrelevant	Same as above																																																
4	ESIZE = 5 DSIZE = 4 EMPTAB <table><tr><td>JONES</td><td>M</td><td>D42</td><td>21,000.00</td></tr><tr><td>WARNS</td><td>M</td><td>D95</td><td>12,000.00</td></tr><tr><td>LORIN</td><td>E</td><td>D42</td><td>10,000.00</td></tr><tr><td>TOY</td><td>E</td><td>D95</td><td>16,000.00</td></tr><tr><td>SMITH</td><td>E</td><td>D32</td><td>14,000.00</td></tr></table> DEPTTAB <table><tr><td>D42</td><td>10,000.00</td></tr><tr><td>D32</td><td>8,000.00</td></tr><tr><td>D95</td><td>10,000.00</td></tr><tr><td>D44</td><td>10,000.00</td></tr></table>	JONES	M	D42	21,000.00	WARNS	M	D95	12,000.00	LORIN	E	D42	10,000.00	TOY	E	D95	16,000.00	SMITH	E	D32	14,000.00	D42	10,000.00	D32	8,000.00	D95	10,000.00	D44	10,000.00	ERRCODE = 2 ESIZE, DSIZE, and DEPTTAB are unchanged EMPTAB <table><tr><td>JONES</td><td>M</td><td>D42</td><td>21,100.00</td></tr><tr><td>WARNS</td><td>M</td><td>D95</td><td>12,100.00</td></tr><tr><td>LORIN</td><td>E</td><td>D42</td><td>10,200.00</td></tr><tr><td>TOY</td><td>E</td><td>D95</td><td>16,100.00</td></tr><tr><td>SMITH</td><td>E</td><td>D32</td><td>14,000.00</td></tr></table>	JONES	M	D42	21,100.00	WARNS	M	D95	12,100.00	LORIN	E	D42	10,200.00	TOY	E	D95	16,100.00	SMITH	E	D32	14,000.00
JONES	M	D42	21,000.00																																															
WARNS	M	D95	12,000.00																																															
LORIN	E	D42	10,000.00																																															
TOY	E	D95	16,000.00																																															
SMITH	E	D32	14,000.00																																															
D42	10,000.00																																																	
D32	8,000.00																																																	
D95	10,000.00																																																	
D44	10,000.00																																																	
JONES	M	D42	21,100.00																																															
WARNS	M	D95	12,100.00																																															
LORIN	E	D42	10,200.00																																															
TOY	E	D95	16,100.00																																															
SMITH	E	D32	14,000.00																																															

V V ->



Exemplo: função BONUS

Cobertura de múltiplas condições

- Alguns problemas permanecem:
 - Retorno = 0 não testado
 - Defeitos de inicialização de variáveis poderiam não ser detectados (LSALARY = \$15000,01)
 - Defeitos em expressões de comparação poderiam não ser detectados:
SALARY(K) > LSALARY em vez de SALARY(K) >= LSALARY
 - Defeitos ao acessar último registro das tabelas não seriam detectados
- Conclusão: a cobertura de múltiplas condições é superior aos outros critérios de cobertura de código, porém não é suficiente como único meio de derivar testes de unidade



Exemplo: função BONUS

Próximo passo: suplementar com testes caixa-preta

Valores de entrada, aplicando análise de valor limite:

1. EMPTAB tem 1 registro.
2. EMPTAB tem o número máximo de registros.
3. EMPTAB tem 0 registros.
4. DEPTTAB tem 1 registro.
5. DEPTTAB tem o número máximo de registros.
6. DEPTTAB tem 0 registros.
7. Um departamento elegível possui 1 funcionário.
8. Um departamento elegível tem o número máximo funcionários (máximo de registros da tabela).
9. Um departamento elegível não tem funcionários.
10. Todos os departamentos do DEPTTAB têm o mesmo valor de venda.
11. O departamento elegível é a primeira entrada em DEPTTAB.
12. O departamento elegível é a última entrada em DEPTTAB.
13. Um funcionário elegível é a primeira entrada em EMPTAB.
14. Um funcionário elegível é a última entrada em EMPTAB.
15. Um funcionário elegível é um gerente.
16. Um funcionário elegível não é um gerente.
17. Um funcionário elegível que não seja gerente recebe um salário de \$ 14.999,99.
18. Um funcionário elegível que não seja gerente recebe um salário de \$ 15.000.
19. Um funcionário elegível que não seja gerente recebe um salário de \$ 15.000,01.

Os limites de saída são os seguintes:

20. ERRCODE = 0
21. ERRCODE = 1
22. ERRCODE = 2
23. O salário aumentado de um funcionário elegível é \$ 299.999,99.

Baseado em Error-Guessing:

24. Um departamento com max vendas sem funcionários é seguido em DEPTTAB por outro departamento max vendas com funcionários.



Exemplo: função BONUS

Próximo passo: suplementar com testes caixa-preta

A partir das situações indentificadas:

1. Excluir situações impraticáveis e que representam situações que nunca ocorrerão (ex: situações 2, 5 e 8).
2. Comparar as situações remanescentes com os casos de teste existentes para identificar as situações de valor limite não cobertas.
 - Situações 1, 4, 7, 10, 14, 17, 18, 19, 20, 23, and 24 requerem casos de teste além dos definidos via caixa-branca.
3. Adicionar novos casos de teste para as situações identificadas (o menor conjunto de casos de teste necessário para cobrir todas as situações).
 - Obs.: não é recomendado alterar os casos de teste existentes, pois isso poderia comprometer inadvertidamente a cobertura de múltiplas condições.

Exemplo: função BONUS

***Próximo passo:
suplementar com testes caixa-preta***

Situações 7, 10, 14, 17, 18, 19 e 20 →

Situações 1, 4 e 23 →

Situação 24 →

Test case	Input	Expected output																												
5	ESIZE = 3 DSIZE = 2 EMPTAB<table><tr><td>ALLY</td><td>E</td><td>D36</td><td>14,999.99</td></tr><tr><td>BEST</td><td>E</td><td>D33</td><td>15,000.00</td></tr><tr><td>CELTO</td><td>E</td><td>D33</td><td>15,000.01</td></tr></table> DEPTTAB<table><tr><td>D33</td><td>55,400.01</td></tr><tr><td>D36</td><td>55,400.01</td></tr></table>	ALLY	E	D36	14,999.99	BEST	E	D33	15,000.00	CELTO	E	D33	15,000.01	D33	55,400.01	D36	55,400.01	ERRCODE = 0 ESIZE, DSIZE, and DEPTTAB are unchanged EMPTAB<table><tr><td>ALLY</td><td>E</td><td>D36</td><td>15,199.99</td></tr><tr><td>BEST</td><td>E</td><td>D33</td><td>15,100.00</td></tr><tr><td>CELTO</td><td>E</td><td>D33</td><td>15,100.01</td></tr></table>	ALLY	E	D36	15,199.99	BEST	E	D33	15,100.00	CELTO	E	D33	15,100.01
ALLY	E	D36	14,999.99																											
BEST	E	D33	15,000.00																											
CELTO	E	D33	15,000.01																											
D33	55,400.01																													
D36	55,400.01																													
ALLY	E	D36	15,199.99																											
BEST	E	D33	15,100.00																											
CELTO	E	D33	15,100.01																											
6	ESIZE = 1 DSIZE = 1 EMPTAB<table><tr><td>CHIEF</td><td>M</td><td>D99</td><td>99,899.99</td></tr></table> DEPTTAB<table><tr><td>D99</td><td>99,000.00</td></tr></table>	CHIEF	M	D99	99,899.99	D99	99,000.00	ERRCODE = 0 ESIZE, DSIZE, and DEPTTAB are unchanged EMPTAB<table><tr><td>CHIEF</td><td>M</td><td>D99</td><td>99,999.99</td></tr></table>	CHIEF	M	D99	99,999.99																		
CHIEF	M	D99	99,899.99																											
D99	99,000.00																													
CHIEF	M	D99	99,999.99																											
7	ESIZE = 2 DSIZE = 2 EMPTAB<table><tr><td>DOLE</td><td>E</td><td>D67</td><td>10,000.00</td></tr><tr><td>FORD</td><td>E</td><td>D22</td><td>33,333.33</td></tr></table> DEPTTAB<table><tr><td>D66</td><td>20,000.00</td></tr><tr><td>D67</td><td>20,000.00</td></tr></table>	DOLE	E	D67	10,000.00	FORD	E	D22	33,333.33	D66	20,000.00	D67	20,000.00	ERRCODE = 2 ESIZE, DSIZE, and DEPTTAB are unchanged EMPTAB<table><tr><td>DOLE</td><td>E</td><td>D67</td><td>10,000.00</td></tr><tr><td>FORD</td><td>E</td><td>D22</td><td>33,333.33</td></tr></table>	DOLE	E	D67	10,000.00	FORD	E	D22	33,333.33								
DOLE	E	D67	10,000.00																											
FORD	E	D22	33,333.33																											
D66	20,000.00																													
D67	20,000.00																													
DOLE	E	D67	10,000.00																											
FORD	E	D22	33,333.33																											

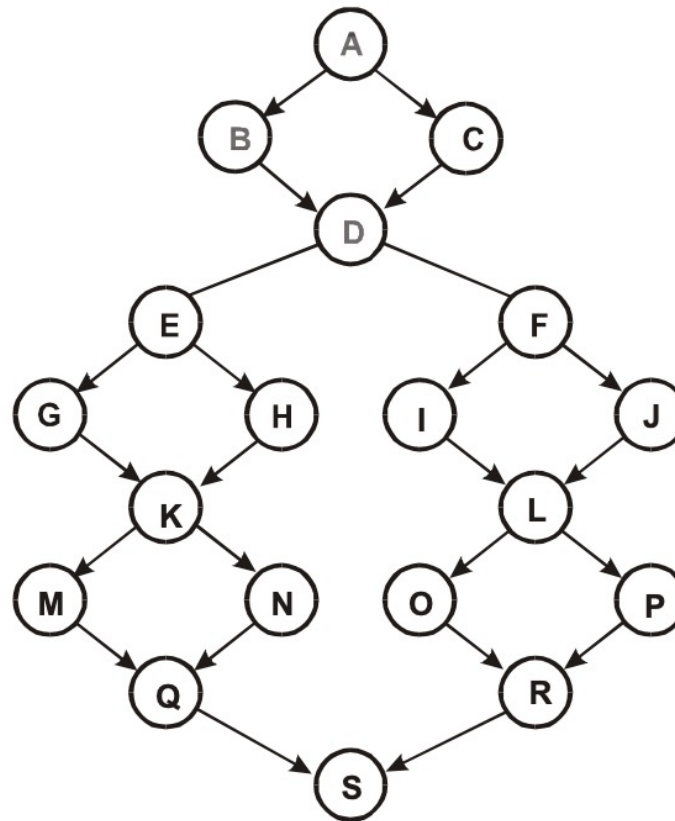
Critério de McCabe

- Critérios baseados na complexidade
 - Critério de McCabe, utiliza **complexidade** ciclomática para derivar os casos de teste
 - O valor da complexidade ciclomática C oferece um limite máximo para o número de **caminhos linearmente independentes e sem loops (caminhos base)**
 - **$C = E - N + 2$** , tal que E é o número de arestas (*edges*) e N é o número de Nós (*nodes*) do GFC
 - Representa o **número de casos de teste** que devem ser criados para garantir a cobertura de todas as instruções e decisões de um programa
 - Oferece uma **medida quantitativa da dificuldade** de conduzir os testes e uma indicação da confiabilidade final

Critério de McCabe

$C = \text{edges} - \text{nodes} + 2$

$C = 24 - 19 + 2 = 7$



Caminhos Base

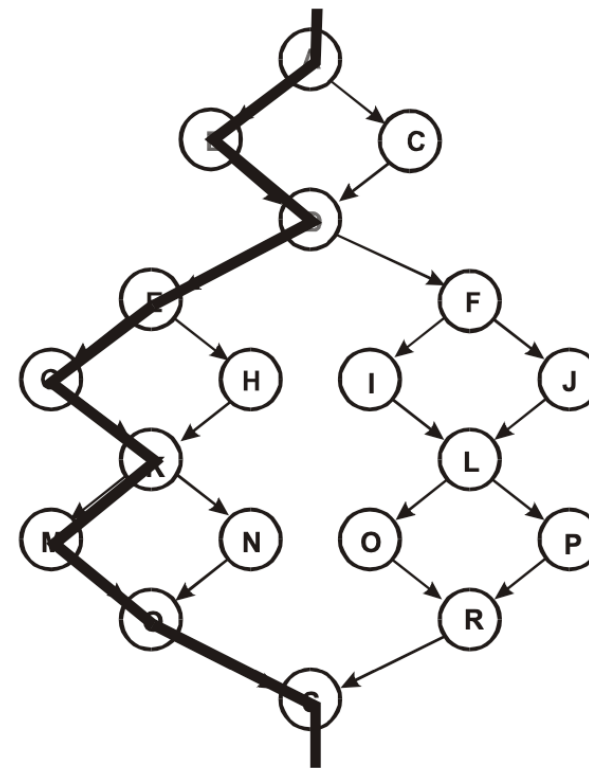
- A complexidade ciclomática define o número mínimo de conjuntos de caminhos independentes livres de *loop* (caminho básico).
- Usa a análise da topologia do grafo para identificar casos de teste.
- Um caminho independente é um fluxo do programa que inclui ao menos um novo conjunto de comandos (nó) ou uma nova decisão (aresta).
- O conjunto de caminhos base inclui todos os nós e arestas do GFC.
- Casos de testes que exercitem todos os caminhos básicos do conjunto também executam todas as instruções e todas as decisões do programa.
- Um conjunto de caminhos base não é único para o grafo!

Caminhos Base

Procedimento para identificar os caminhos base:

1. Escolha um caminho base. Este caminho pode ser um caminho típico de execução (caminho feliz).

1º caminho base



ABDEGKM QS

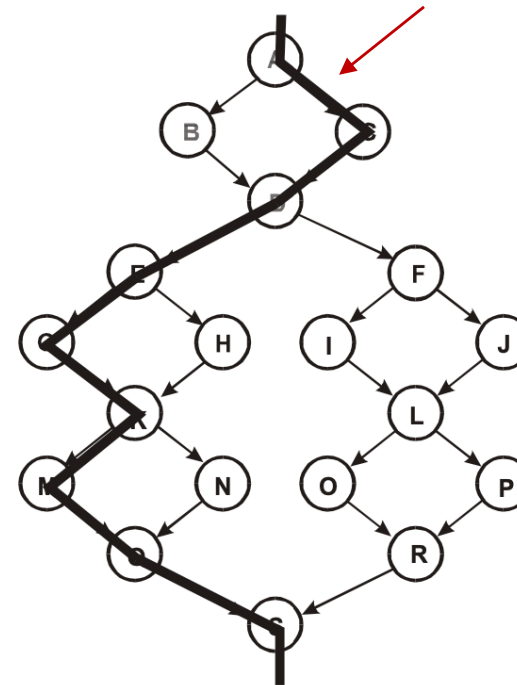


Caminhos Base

Procedimento para identificar os caminhos base:

1. Escolha um caminho base. Este caminho pode ser um caminho típico de execução (caminho feliz).
2. Para escolher o próximo caminho, mude a saída da primeira decisão do caminho base, mantendo o máximo possível o restante do caminho inalterado.

2º caminho base



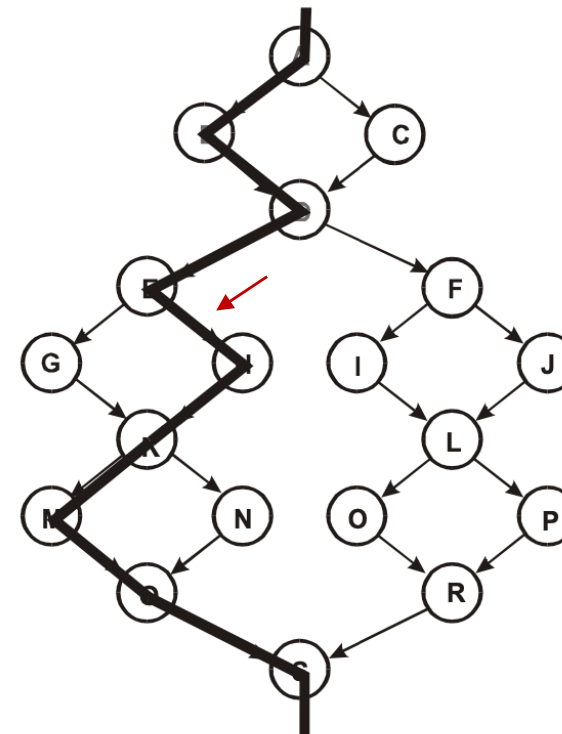
ACDEGKMQS

Caminhos Base

Procedimento para identificar os caminhos base:

1. Escolha um caminho base. Este caminho pode ser um caminho típico de execução (caminho feliz).
2. Para escolher o próximo caminho, mude a saída da primeira decisão do caminho base, mantendo o máximo possível o restante do caminho inalterado.
3. Para gerar o terceiro caminho, comece novamente no caminho base, mas altere a segunda decisão em vez da primeira.

3º caminho base



ABDEHKMQS

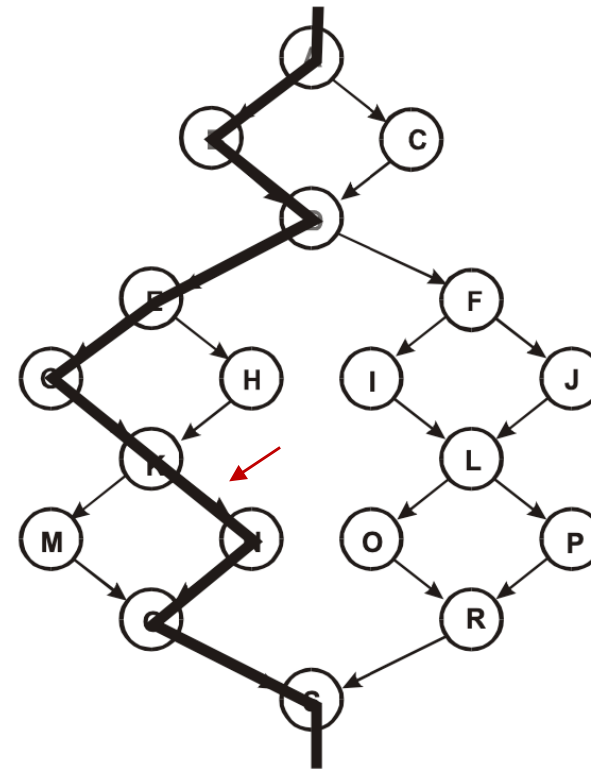


Caminhos Base

Procedimento para identificar os caminhos base:

1. Escolha um caminho base. Este caminho pode ser um caminho típico de execução (caminho feliz).
2. Para escolher o próximo caminho, mude a saída da primeira decisão do caminho base, mantendo o máximo possível o restante do caminho inalterado.
3. Para gerar o terceiro caminho, comece novamente no caminho base, mas altere a segunda decisão em vez da primeira.
4. **Para gerar o quarto caminho, comece no caminho base e mude a terceira decisão em vez da segunda. Continue variando cada decisão, uma a uma, até que o fim do grafo seja alcançado.**

4º caminho base



ABDEGKNQS

Caminhos Base

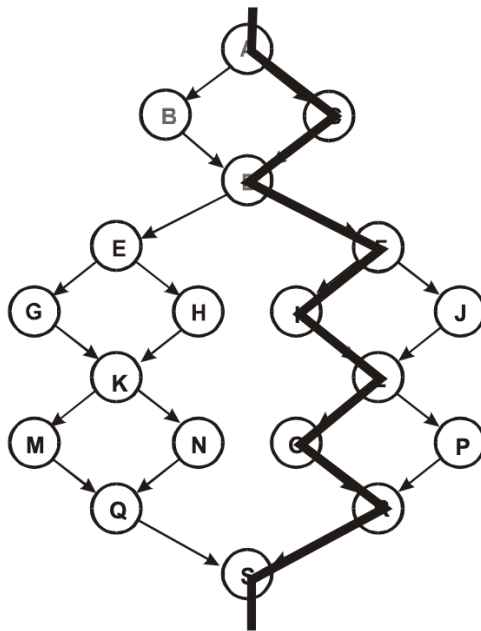
Procedimento para identificar os caminhos base:

1. Escolha um caminho base. Este caminho pode ser um caminho típico de execução (caminho feliz).
2. Para escolher o próximo caminho, mude a saída da primeira decisão do caminho base, mantendo o máximo possível o restante do caminho inalterado.
3. Para gerar o terceiro caminho, comece novamente no caminho base, mas altere a segunda decisão em vez da primeira.
4. Para gerar o quarto caminho, comece no caminho base e mude a terceira decisão em vez da segunda. Continue variando cada decisão, uma a uma, até que o fim do grafo seja alcançado.
5. **Agora que todas as decisões do caminho base foram contempladas, prossegue-se para o segundo caminho contemplando todas as decisões, uma a uma. Esse padrão continua até que o conjunto de caminhos básicos esteja completo.**



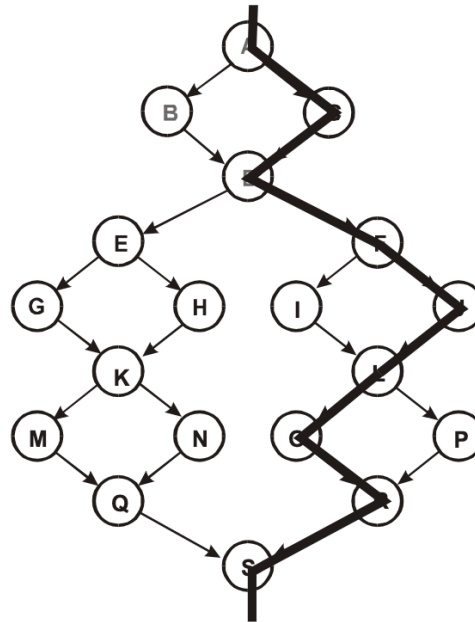
Caminhos Base

5º caminho base



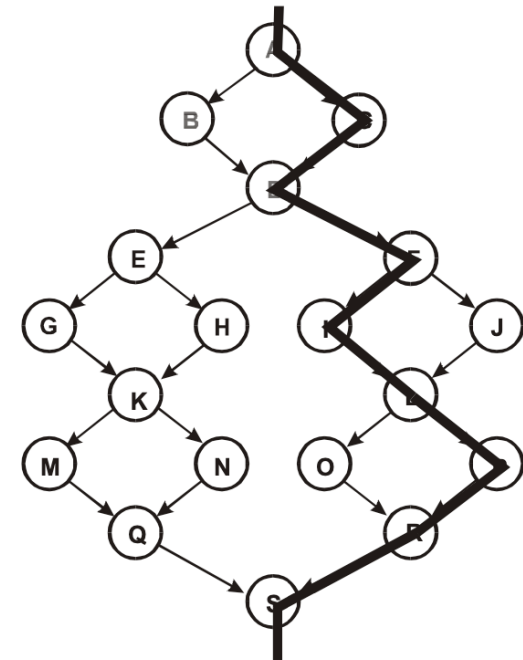
ACDFILORS

6º caminho base



ACDFJLORS

7º caminho base



ACDFILPRS



Caminhos Base

- O conjunto de caminhos base para o grafo é:
 1. ABDEGKM QS
 2. ACDEGKM QS
 3. ABDEHKM QS
 4. ABDEGKN QS
 5. ACDFILORS
 6. ACDFJLORS
 7. ACDFILPRS
- Deve ser criado um caso de teste para cada caminho
- Este conjunto de testes garante a cobertura de comandos e decisões
- Observar que o conjunto de caminhos não é único



Exemplo – Caminhos Básicos

Algoritmo de Busca Binária

```
int binsearch(int X, int V[], int n){  
    int low, high, mid;  
    low = 0;  
    high = n - 1;  
    while (low <= high) {  
        mid = (low + high)/2;  
        if (X < V[mid])  
            high = mid - 1;  
        else if (X > V[mid])  
            low = mid + 1;  
        else  
            return mid;  
    }  
    return -1;  
}
```

1

2

3

4

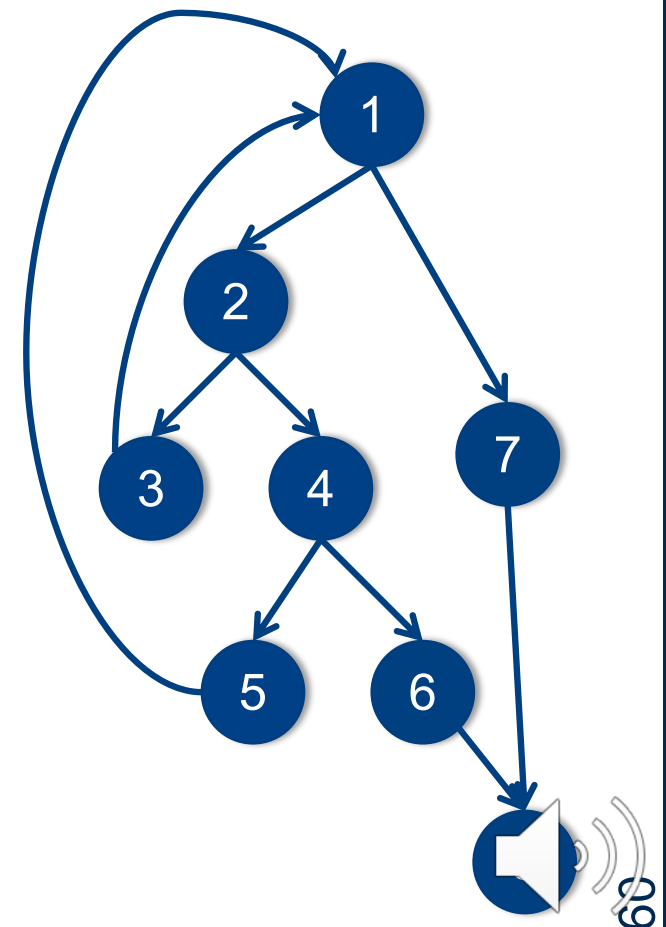
5

6

7

8

Grafo de Fluxo de Controle



Exemplo – Caminhos Básicos

- $C = E - N + 2$, tal que E é o número de arcos e N é o número de Nós do GFC G
- $C = 10 - 8 + 2 = 4$
- Caminhos Básicos:
 - C1) 1, 2, 4, 6, 8
 - C2) 1, 7, 8
 - C3) 1, 2, 3, 1, 2, 4, 6, 8
 - C4) 1, 2, 4, 5, 1, 2, 4, 6, 8



Caixa Branca - Critérios

- Critérios baseados no fluxo de dados
 - mesmo para programas pequenos, o teste baseado unicamente no fluxo de controle não é eficaz para revelar a presença até mesmo de erros simples e triviais
 - utilizam informações do fluxo de dados do programa para determinar os requisitos de teste
 - exploram as interações que envolvem definições de variáveis e referências a tais definições para estabelecerem os requisitos de teste
 - Exemplos:
 - Critérios de Rapps e Weyuker
 - Critérios Potenciais-Usos



Teste Caixa-Cinza

- Testes que envolvem entradas e saídas, mas projetados considerando informações sobre o código ou operação do programa de uma forma que estaria normalmente fora do escopo de visão de um testador. [Cem Karner]
- Teste elaborado com base no conhecimento do algoritmo, estados internos, arquiteturas, ou outras descrições de alto nível sobre o comportamento do programa. [Doug Hoffman]



Conclusões

- Para derivar um bom conjunto de casos de teste é necessário combinar elementos de ambas abordagens caixa preta e caixa branca
- Cada método contribui com um conjunto útil de casos de teste
- Recomenda-se desenvolver casos de teste utilizando métodos caixa-preta e suplementar com métodos caixa branca, conforme for necessário



Referências

- Myers, G. J. et al, The Art of Software Testing. Ed. John Wiley & Sons, 2012.
- Critério de Caminhos Base
 - Copeland, Lee. A Practitioner's Guide to Software Test Design. Norwood, MA, USA: Artech House, 2003. Capítulo 10.
 - Livro disponível no EBookCentral
- Aniche, M., Effective Software Testing: A developer's Guide. Manning, 2021.

